Pima Indians Diabetes Prediction Accuracy and Tuning

Jermaine Pau Malveda
College of Engineering, Architecture, and Technology
De La Salle University - Dasmariñas
Dasmarinas City, Philippines
mjt0334@dlsud.edu.ph

***Abstract*— This paper dives into an exploration of diverse machine learning models aimed emphasizing Random Forest Classifier at accurately predicting the likelihood of diabetes in individuals by using Accuracy, F1 score and utilizing the Pima Indian dataset as a basis that is composed of 9 attributes of 768 female patients. Through the analysis of this dataset, the researcher seeks to identify and compare the efficacy of different machine learning algorithms in providing reliable predictions for diabetes diagnosis.**

Keywords—Random Forest Classifier, Accuracy, F1 Score, Diabetes, Prediction, Machine Learning Models

I. Introduction

The aim of this paper is to precisely predict the presence of diabetes in patients within the Pima Indian database. According to Deepti Sisodia (2018), prior investigations have underscored the effectiveness of machine learning methodologies in precisely diagnosing diverse diseases, as supported by numerous research initiatives.

These methods encompass J48, SVM, Naive Bayes, Decision Tree, and Decision Table. Such findings and recommendations are poised to aid researchers in exploring additional avenues for advancement, particularly in the contemporary integration of AI. As per Akihiro Nomura (2021), there has been a growing adoption of artificial intelligence (AI) within the domain of diabetes management. The US FDA has approved numerous AI and machine learning (ML) driven medical devices for various functions including automated retinal screening, clinical diagnosis assistance, and patient self-care.

By leveraging the diagnostic measures provided in the dataset, the researcher will utilize the predictor values: Outcome 1, indicating they have diabetes, and Outcome 0, indicating they do not have diabetes, as the foundation for our machine learning models. This will facilitate accurate predictions of diabetes status.

II. Data Pre-Processing

A. Data Description

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

a. Raw Pima Indians Diabetes Dataset

Utilizing the Pima Indian dataset, It is composed of 9 attributes (columns) including the target variable “Outcome” 1 and 0 out of 768 female patients (rows).

```
from sklearn.impute import SimpleImputer

# Checking for missing values
print(df.isnull().sum())

# Impute missing values using mean strategy
imputer = SimpleImputer(strategy='mean')
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
```

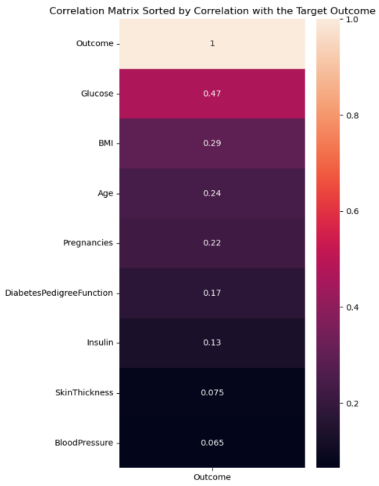
b. Replacing Nan Values with educated guess

```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64

After SimpleImputer
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

```
# Checking for missing values after handling
print("\nAfter SimpleImputer")
print(df_imputed.isnull().sum())
```

c. Checking for Nan Values.

B. Correlation Matrix



a. Evaluating the correlation of target “Outcome” and feature values

The Correlation matrix explains that the BloodPressure and SkinThickness values is very close to 0, it will be dropped because they have a low correlation to the target outcome.

```
#Drop SkinThickness and BloodPressure
df = df.drop(['SkinThickness', 'BloodPressure',], axis=1)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 7 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   Insulin                   768 non-null    int64
 3   BMI                       768 non-null    float64
 4   DiabetesPedigreeFunction  768 non-null    float64
 5   Age                       768 non-null    int64
 6   Outcome                   768 non-null    int64
dtypes: float64(2), int64(5)
memory usage: 42.1 KB
```

b. Replacing Nan Values with educated guess

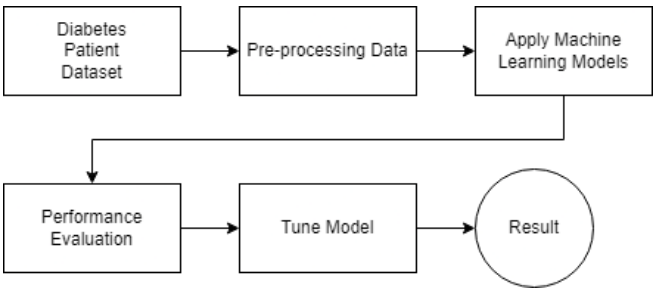
Removing the low-correlated values will enable the machine learning models to perform better due to the cleaner dataset. Subsequently, scaling will be applied to ensure uniformity across all features in the clean dataset, followed by splitting into testing and training sets for building the machine learning models.

```
#The data will now be split to 30% testing and 70% training
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

c. Splitting and scaling data

III. METHODOLOGY

The study will solely focus on classification models since the dataset does not involve ongoing numerical values suitable for regression analysis. The researcher will employ a balanced approach by integrating established classification methods alongside tuning techniques.



a. Framework of the flow of methodology

IV. TRAINING AND EVALUATING CLASSIFICATION MODELS

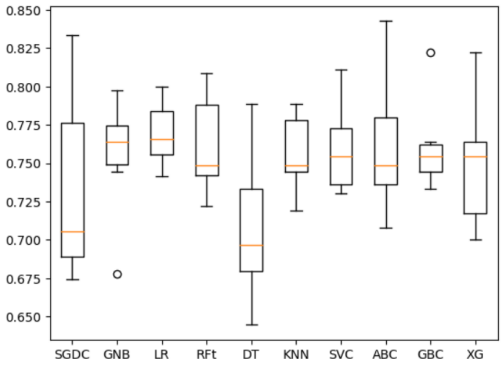
Selected and tested Machine Learning Models

- SGD Classifier (SGC)
- GuassianNB (GNB)
- Logistic Regression (LR)
- Random Forest Classifier (Rft)
- Decision Tree Classifier (DT)
- Kneighbors Classifier (KNN)
- Support Vector Classifier (SVC)
- AdaBoost Classifier (ABC)
- Gradient Boosting Classifier (GBC)
- Xg Boost Classifier (XGB)

A. *Cross Validation*

In this Cross Validation boxplot, it is evident that GaussianNB exhibits the highest minimum value among the Cross Validation scores, indicating strong performance.

However, it is important to note that other machine learning models also display minimum scores that are relatively close to GaussianNB's, suggesting comparable performance across the models.



d. Boxplot depicting Cross Validation Scores of the Models

B. *Accuracy Score*

It's important to consider that accuracy is a crucial metric, as it reflects the model's ability to perform well. Among the machine learning models used, SVC achieves the highest accuracy score (77.06%), followed closely by GNB (75.76%) and Rft (75.32%).

This suggests that GNB and Rft demonstrate consistency and represent strong candidates for model selection, as they exhibit favorable performance in both Cross Validation and Accuracy Score analyses.

```
SGDC Test Set Accuracy: 68.40%
GNB Test Set Accuracy: 75.76%
LR Test Set Accuracy: 73.59%
RFt Test Set Accuracy: 75.32%
DT Test Set Accuracy: 71.43%
KNN Test Set Accuracy: 68.83%
SVC Test Set Accuracy: 77.06%
ABC Test Set Accuracy: 75.32%
GBC Test Set Accuracy: 72.73%
XG Test Set Accuracy: 70.56%
```

e. Report of Accuracy Scores of the selected models

C. *Confusion Matrix*

After assessing accuracy testing, it became evident that it might not be the optimal basis for model selection. Consequently, the researcher turned to the confusion matrix, which offers four performance measures, including recall and F1 score, for a more comprehensive evaluation.

Given the importance of minimizing false negatives in our dataset where one class is significantly more prevalent, we prioritize the F1 score as it balances precision and recall, crucial for accurately identifying diabetic patients.

```
RFt:

[[115  36]
 [ 25  55]]
              precision    recall  f1-score   support

           0       0.82      0.76      0.79       151
           1       0.60      0.69      0.64        80

    accuracy                           0.74       231
   macro avg       0.71      0.72      0.72       231
weighted avg       0.75      0.74      0.74       231

Recall: 0.6875
Accuracy: 0.7359307359307359
F1 score: 0.6432748538011696
```

```
GNB:

[[122  29]
 [ 27  53]]
              precision    recall  f1-score   support

           0       0.82      0.81      0.81       151
           1       0.65      0.66      0.65        80

    accuracy                           0.76       231
   macro avg       0.73      0.74      0.73       231
weighted avg       0.76      0.76      0.76       231

Recall: 0.6625
Accuracy: 0.7575757575757576
F1 score: 0.654320987654321
```

f. Confusion matrix of the highest F1 score, Rft and GNB

In the confusion matrix the basis is the highest F1 score which is Rft (64%) and GNB (65%).

D. *Building Models*

```
gnb_model = GaussianNB()
gnb_model.fit(X_train_scaled, y_train)

# Calculate training and testing accuracies
train_accuracy = accuracy_score(y_train, gnb_model.predict(X_train_scaled))*100
test_accuracy = accuracy_score(y_test, gnb_model.predict(X_test_scaled))*100

print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)
```

```
Training Accuracy: 76.72253258845437
Testing Accuracy: 75.75757575757575
```

```
rfm = RandomForestClassifier(n_jobs=-1, random_state=42)

# Fit the model to the training data
rfm.fit(X_train, y_train)

# Make predictions on the training data
y_train_pred = rfm.predict(X_train)

# Make predictions on the testing data
y_test_pred = rfm.predict(X_test)

# Calculate training and testing accuracies
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

# Print the accuracies
print("Training Accuracy: {:.2f}%".format(train_accuracy * 100))
print("Testing Accuracy: {:.2f}%".format(test_accuracy * 100))
```

```
Training Accuracy: 100.00%
Testing Accuracy: 75.32%
```

g. Building Model Training and Testing Accuracy

In the process of building the models, GaussianNB and Random Forest Classifier consistently achieved the highest training and testing accuracy. This demonstrates their reliability and consistency across both training and testing phases.

E. *Summary and Model Selection*

The researcher has decided to choose the Random Forest Classifier as our model of choice. Through thorough testing, it consistently maintains on of the top position among the models considered. As suggested by Sruthi (2024), its ability to capture complex relationships within the data, thanks to its composition of decision trees, makes it particularly suitable for predicting diabetes. Unlike GaussianNB, which only accounts for independent features, Random Forest Classifier acknowledges the interrelation among features, crucial for accurate predictions in this context which is diabetes.

V. Model Tuning

In fine-tuning the model, we will employ GridSearchCV to adjust hyperparameters.

The parameters to be tuned include: n_estimators, which determines the number of trees in the forest; max_depth, setting the maximum depth of each tree; min_samples_split, defining the minimum number of samples required to split a node; and min_samples_leaf, specifying the minimum number of samples required to be at a leaf node.

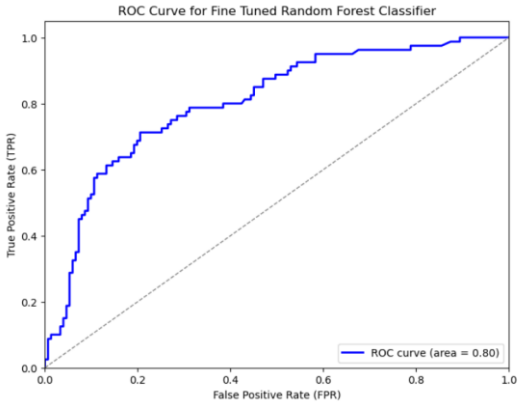
```
param_grid = {
    'n_estimators': [200,300,400],
    'max_depth': [None, 15, 20],
    'min_samples_split': [2, 3],
    'min_samples_leaf': [1, 2],
    'max_features': ['sqrt', 'log2']
}
```

a. Parameters of Random Forest Classifier tuning

Results:

- Best Parameters: {'max_depth': 15, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 400}
- Accuracy after tuning: 76.32%

After hyperparameter tuning, the best parameters are applied to the prediction process. Subsequently, we proceed to visualize the classification report and ROC curve to assess the model's performance.

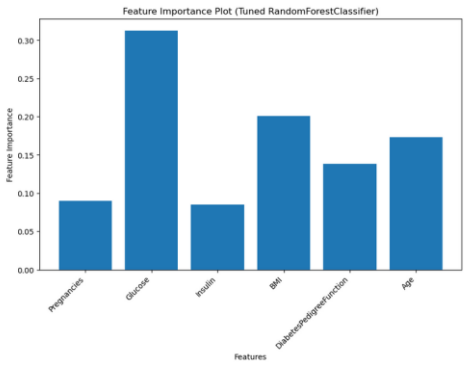



```
Classification Report for Tuned RandomForestClassifier:
              precision    recall  f1-score   support

           0       0.84      0.79      0.82       151
           1       0.65      0.71      0.68        80

    accuracy                           0.77       231
   macro avg       0.74      0.75      0.75       231
weighted avg       0.77      0.77      0.77       231
```

b. ROC Curve and Classification Report of Tuned Model

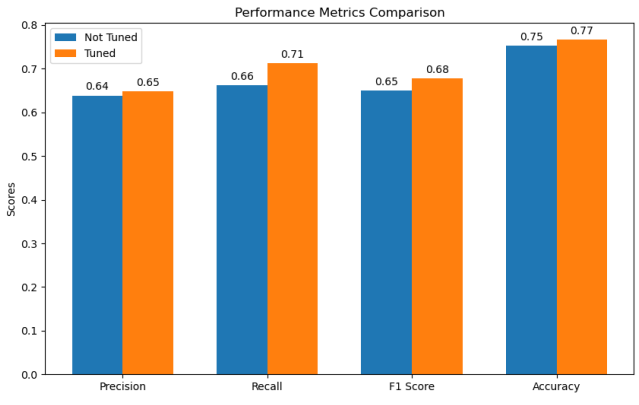
VI. Results and Discussion of Tuned Model

A. *Feature Importance Plot*



a. Feature Importance after tuning the model

It is notable that Glucose significantly influences the other features, which is reasonable considering its pivotal role in predicting diabetes, as high glucose levels are characteristic of the condition.



b. Performance of metrics Comparison of Tuned and Not Tuned Model

The performance evaluation of both the trained and untrained Random Forest Classifier models shows an accuracy range of 74-75%. However, following hyperparameter tuning, the model's accuracy improved by 1.3%. Additionally, Precision increased by 0.01, Recall by 0.05, and the crucial F1 score by 0.03. With an increase in accuracy by 0.02, it can be concluded that all metrics experienced improvement.

VII. LIMITATIONS AND FUTURE WORK

One limitation arises from the size and composition of the dataset, which might not offer adequate information to ensure the absolute reliability of predictive models. To address this, expanding the dataset or acquiring additional data could potentially enhance the robustness of the models. Moreover, while Random Forest has shown promise, exploring alternative machine learning algorithms and ensemble techniques could further improve performance.

Additionally, a limitation lies in the use of existing models and algorithms, which could be addressed by combining different models to enhance accuracy. Future work needs to involve adding more diverse data to the dataset to better capture the essence of diabetes prediction. Furthermore, certain machine learning models may not require extensive parameter tuning, allowing focus on other aspects such as feature selection or ensemble methods. Exploring alternative algorithms may also lead to improved results.

VIII. CONCLUSION

In conclusion, preprocessing data is crucial, encompassing tasks like scaling and removing low correlation features to balance data. These steps significantly enhance model performance and accuracy, influencing your approach to the dataset and potentially increasing its accuracy. Moreover, Following hyperparameter tuning with GridSearchCV, the Random Forest Classifier demonstrated a notable 1.3% improvement. Lastly, future works should prioritize acquiring larger and diverse datasets to facilitate more robust model training, optimization, and improved prediction accuracy for diabetes.

REFERENCES

[1] *Diabetes Prediction* . (n.d.). Retrieved March 12, 2024, from https://kaggle.com/code/zabihullah18/diabetes-prediction

[2] *Gaussian Naive Bayes (GaussianNB) implementation · scikit-learn/scikit-learn · Discussion #19141*. (n.d.). GitHub. Retrieved March 12, 2024, from https://github.com/scikit-learn/scikit-learn/discussions/19141

[3] *How to Choose Right Metric for Evaluating ML Model*. (n.d.). Retrieved March 12, 2024, from https://kaggle.com/code/vipulgandhi/how-to-choose-right-metric-for-evaluating-ml-model

[4] Nomura, A., Noguchi, M., Kometani, M., Furukawa, K., & Yoneda, T. (2021). Artificial Intelligence in Current Diabetes Management and Prediction. *Current Diabetes Reports*, *21*(12), 61. https://doi.org/10.1007/s11892-021-01423-2

[5] *(Data set used)Pima Indians Diabetes Database*. (n.d.). Retrieved March 12, 2024, from https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database

[6] R, S. E. (2021, June 17). Understand Random Forest Algorithms With Examples (Updated 2024). *Analytics Vidhya*. https://www.analyticsvidhya.com/blog/2021/06/understanding-random-forest/

[7] Sisodia, D., & Sisodia, D. S. (2018). Prediction of Diabetes using Classification Algorithms. *Procedia Computer Science*, *132*, 1578–1585. https://doi.org/10.1016/j.procs.2018.05.122